



Enterprise Java & Concurrency Engineering

A production-ready training curriculum for full-stack software engineers and backend Java developers. This deck covers four progressive modules — from Java fundamentals and OOP through advanced constructs, I/O, and enterprise-grade multithreading — with executable code examples, theoretical depth, and real-world challenge scenarios at every stage.

4 MODULES

CONCURRENCY-READY

PRODUCTION-GRADE

Curriculum Overview

Four tightly structured modules build on each other progressively, culminating in enterprise concurrency patterns used in production systems.



Module 1 — Java Basics & Logic Building

Class architecture, primitives, String forensics, Scanner input, recursion, and partition generation.



Module 3 — Advanced Constructs, I/O & Modern Java

Enums, lambdas, File I/O, Records, text blocks, switch expressions, and Swing GUI basics.



Module 2 — Intermediate Java, OOP & Collections

Arrays, Wrapper classes, exception handling, Collections Framework, and iteration strategies.



Module 4 — Multithreading & Concurrency

Thread lifecycle, synchronisation, AtomicInteger, ExecutorService, and Producer-Consumer patterns.

Java Fundamentals: Class Architecture & Execution Flow

Every Java programme begins with a `class` definition. The JVM locates the entry point via `public static void main(String[] args)`. Understanding this contract — **public** (accessible by JVM), **static** (no instance required), **void** (no return value) — is non-negotiable in enterprise development. Variable declarations follow strict typing rules, and choosing the correct primitive (`int`, `long`, `double`, `boolean`, `char`) directly impacts memory layout and arithmetic precision.

Key Concepts

- **Class architecture:** fields, constructors, methods, access modifiers
- **main() contract:** JVM entry point signature
- **Primitives vs references:** stack vs heap allocation
- **print vs println:** buffered vs newline-terminated output
- **Arithmetic operators:** `+` `-` `*` `/` `%` and integer division pitfalls

Executable Example

```
public class Fundamentals {  
    static int counter = 0; // class-level field  
  
    public static void main(String[] args) {  
        int a = 10, b = 3;  
        double ratio = (double) a / b; // cast prevents int division  
        System.out.print("Ratio: ");  
        System.out.println(ratio);    // 3.3333...  
        counter++;  
        System.out.println("Counter: " + counter);  
    }  
}
```

❏ Integer division truncates: `10 / 3 == 3`, not 3.33. Always cast to `double` when fractional precision is required.

String Forensics & Conditional Branching

Strings in Java are immutable objects backed by a character array. Mastering the standard String API is essential for parsing, validating, and transforming data in enterprise services. Pair this with structured branching — `if/else` for binary decisions, `switch` for multi-branch dispatch — and you have the backbone of business logic engines.

Core String Methods

```
String s = "Hello, World!";
s.toUpperCase() // "HELLO, WORLD!"
s.toLowerCase() // "hello, world!"
s.length()      // 13
s.charAt(7)     // 'W'
s.substring(7, 12) // "World"
s.contains("World") // true
s.replace("World", "Java") // "Hello, Java!"
```

Escape Sequences

```
"\n" // newline  "\t" // tab
 "\"" // quote   "\\" // backslash
```

Branching Example

```
String grade = "B";
switch (grade) {
    case "A" -> System.out.println("Distinction");
    case "B" -> System.out.println("Merit");
    case "C" -> System.out.println("Pass");
    default -> System.out.println("Review needed");
}
```

// if/else equivalent

```
int score = 72;
if (score >= 90) System.out.println("A");
else if (score >= 75) System.out.println("B");
else System.out.println("C");
```

Scanner Input, Varargs, Recursion & Backtracking

`java.util.Scanner` reads from `stdin` token-by-token using `next()`, `nextLine()`, and `nextInt()`. Varargs (`int... nums`) allow methods to accept a variable number of arguments, useful in utility and logging APIs. Recursion solves problems by reducing them to smaller sub-problems; backtracking augments recursion by abandoning invalid paths and exploring alternatives — the foundation of partition generation and constraint-satisfaction algorithms.

Scanner & Varargs

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter name: ");
String name = sc.nextLine();

static int sum(int... nums) {
    int total = 0;
    for (int n : nums) total += n;
    return total;
}
// sum(1,2,3,4) → 10
```

Recursive String Reversal

```
static String reverse(String s) {
    if (s.isEmpty()) return s;
    return reverse(s.substring(1))
        + s.charAt(0);
}
// reverse("Java") → "avaJ"

// Partition generation (backtracking)
static void partition(String s, List<String> path) {
    if (s.isEmpty()) { System.out.println(path); return; }
    for (int i = 1; i <= s.length(); i++) {
        path.add(s.substring(0, i));
        partition(s.substring(i), path);
        path.remove(path.size() - 1); // backtrack
    }
}
```

- ❏ **Challenge:** Extend the partition algorithm to return only palindromic substrings. Consider memoising palindrome checks for strings longer than 15 characters.

OOP Deep Dive: Arrays, Wrappers & Exception Handling

Multi-dimensional arrays model tabular and matrix data. Wrapper classes (Integer, Double, Boolean) bridge primitives with the Collections Framework via **autoboxing** (primitive → object) and **unboxing** (object → primitive). Exception handling with try-catch-finally isolates failure domains; custom domain exceptions carry business context. Inheritance enables code reuse whilst field shadowing and method overriding allow subclasses to specialise behaviour.

Arrays

```
int[][] matrix = new int[3][3];
```

Access: matrix[row][col]

2D iteration requires nested loops. Arrays.sort() and Arrays.copyOf() are utility staples.

Autoboxing

Integer x = 42; — compiler inserts Integer.valueOf(42). Unboxing: int y = x; inserts x.intValue(). Beware NullPointerException on unboxing null.

Custom Exceptions

```
class InsufficientFundsException
    extends RuntimeException {
    public InsufficientFundsException(
        double amt) {
        super("Insufficient: $" + amt);
    }
}
```

try-catch-finally

The finally block always executes — use it for resource cleanup. Prefer unchecked exceptions for programming errors; checked exceptions for recoverable conditions (I/O, network).

Java Collections Framework: Choosing the Right Structure

Selecting the wrong collection is one of the most common performance mistakes in enterprise Java. Each data structure has distinct time complexities, ordering guarantees, and thread-safety contracts. The table below summarises the critical decision axes.

Collection	Key Methods	Ordering	Duplicates	Use Case
ArrayList	add, get, remove, size	Insertion order	Yes	Random-access list
Stack	push, pop, peek, isEmpty	LIFO	Yes	Undo / call-stack simulation
LinkedList (Queue)	offer, poll, peek	FIFO	Yes	Task queues, BFS
HashMap	put, get, keySet, values, containsKey	None	Keys: No	Fast key-value lookup
HashSet	add, contains, remove, size	None	No	Deduplication, membership

❏ Prefer `ArrayDeque` over `Stack` in modern Java — it is faster and not synchronised. For concurrent access, use `ConcurrentHashMap` and `CopyOnWriteArrayList`.

Iteration Mechanisms & In-Memory Cache Implementation

Four Iteration Strategies

```
List<String> items = List.of("A","B","C");

// 1. Index-based
for (int i = 0; i < items.size(); i++)
    System.out.println(items.get(i));

// 2. Enhanced for-each
for (String item : items)
    System.out.println(item);

// 3. Iterator (safe removal)
Iterator<String> it = items.iterator();
while (it.hasNext()) System.out.println(it.next());

// 4. Lambda forEach
items.forEach(item -> System.out.println(item));
// or method reference:
items.forEach(System.out::println);
```

In-Memory Cache with HashMap

```
class SimpleCache {
    private final Map<String, Object> store
        = new HashMap<>();
    private final List<String> accessLog
        = new ArrayList<>();

    public void put(String key, Object val) {
        store.put(key, val);
        accessLog.add("PUT:" + key);
    }

    public Object get(String key) {
        accessLog.add("GET:" + key);
        return store.getDefault(key, null);
    }

    public List<String> getLog() {
        return Collections.unmodifiableList(accessLog);
    }
}
```

Challenge: Add an LRU eviction policy – evict the least-recently-used entry when the cache exceeds a configurable capacity. Implement a custom `BankAccount` exception hierarchy with `InsufficientFundsException`, `AccountFrozenException`, and a catch-all `BankingException`.

Advanced Constructs: Enums, Lambdas & Functional Interfaces

Enums are type-safe constants ideal for state machines and switch dispatch. Functional interfaces (`@FunctionalInterface`) define a single abstract method – they are the contract that lambda expressions fulfil. Anonymous inner classes provide inline implementations without naming. The ternary operator (`condition ? a : b`) replaces simple if/else. Command-line arguments (`args[0]`) allow runtime parameterisation of enterprise batch jobs.

Enum in Switch

```
enum Status { ACTIVE, INACTIVE,
PENDING }
Status s = Status.ACTIVE;
switch (s) {
    case ACTIVE -> process();
    case INACTIVE -> archive();
    case PENDING -> queue();
}
```

Functional Interface & Lambda

```
@FunctionalInterface
interface Transformer<T> {
    T transform(T input);
}
Transformer<String> upper = s ->
s.toUpperCase();
System.out.println(upper.transform
("hello")); // HELLO
```

Ternary & toString()

```
String label = score >= 60 ? "Pass" :
"Fail";

@Override
public String toString() {
    return "User{name=" + name
+ ", role=" + role + "}";
}
```

File I/O: Reading & Writing Text Files

Enterprise applications routinely read configuration files, audit logs, and data feeds. Java provides several I/O abstractions: `FileReader/BufferedReader` for line-by-line reading, `FileWriter` for character-level writing, and the try-with-resources statement to guarantee stream closure even on exceptions. Always specify a `Charset` (e.g., `StandardCharsets.UTF_8`) to prevent platform-dependent encoding bugs.

Reading a File (Buffered)

```
// try-with-resources auto-closes the reader
try (BufferedReader br = new BufferedReader(
    new FileReader("data.txt",
        StandardCharsets.UTF_8))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    System.err.println("Read error: " + e.getMessage());
}
```

Writing a File (FileWriter)

```
// append=true keeps existing content
try (FileWriter fw = new FileWriter(
    "output.txt", StandardCharsets.UTF_8, true)) {
    fw.write("Audit log entry\n");
    fw.write("Timestamp: " + LocalDateTime.now());
} catch (IOException e) {
    System.err.println("Write error: " + e.getMessage());
}

// Scanner-based reading (simpler for tokens)
try (Scanner sc = new Scanner(
    new File("config.txt"))) {
    while (sc.hasNextLine())
        System.out.println(sc.nextLine());
}
```

📌 Prefer `Files.readAllLines()` and `Files.writeString()` from `java.nio.file.Files` for concise, modern file operations in Java 11+.

Modern Java Features: Records, Text Blocks & Pattern Matching

Java 16–21 introduced a wave of productivity features that reduce boilerplate and improve code expressiveness. **Records** are immutable data carriers — the compiler auto-generates constructor, accessors, `equals`, `hashCode`, and `toString`. **Text blocks** eliminate escape-sequence clutter for JSON/SQL/HTML literals. **Switch expressions** return values. **Pattern matching for instanceof** eliminates redundant casting.

Records & Text Blocks

```
// Record — immutable data carrier
record Employee(String name, int id, double salary) {}

Employee emp = new Employee("Ada", 1001, 95000.0);
System.out.println(emp.name()); // Ada
System.out.println(emp);        // Employee[name=Ada, id=1001,
salary=95000.0]

// Text block — multiline string literal
String json = """
{
  "name": "%s",
  "id": %d
}
""".formatted(emp.name(), emp.id());
```

Pattern Matching & Switch Expression

```
// Pattern matching instanceof (Java 16+)
Object obj = new Employee("Bob", 2, 80000);
if (obj instanceof Employee e) {
    System.out.println("Salary: " + e.salary());
    // no cast needed — e is already Employee
}

// Switch expression returning a value
double bonus = switch (emp.name()) {
    case "Ada" -> emp.salary() * 0.20;
    case "Bob" -> emp.salary() * 0.15;
    default -> emp.salary() * 0.10;
};

// var keyword (local type inference)
var employees = new ArrayList<Employee>();
employees.add(emp);
```

Swing GUI Basics: JFrame, JButton & ActionListener

Swing provides a lightweight, platform-independent GUI toolkit. The core pattern is: create a `JFrame` (window), add components (`JButton`, `JLabel`, `TextArea`), wire event handlers via `ActionListener`, and pack/display. Understanding this event-driven model is directly transferable to JavaFX, Android, and reactive UI frameworks.

Minimal Swing Application

```
import javax.swing.*;
import java.awt.event.*;

public class FileViewer extends JFrame {
    private JTextArea area = new JTextArea(20, 40);
    private JButton load = new JButton("Load File");

    public FileViewer() {
        setTitle("File Content Viewer");
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        add(new JScrollPane(area));
        add(load, "South");

        load.addActionListener(e -> loadFile());
        pack();
        setVisible(true);
    }

    private void loadFile() {
        // read file and set area.setText(content)
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(FileViewer::new);
    }
}
```

Event-Driven Pattern

ActionListener

Interface with `actionPerformed(ActionEvent e)`. Implement via lambda since it is a functional interface.

SwingUtilities.invokeLater

All Swing updates must run on the **Event Dispatch Thread (EDT)**. `invokeLater` schedules safely.

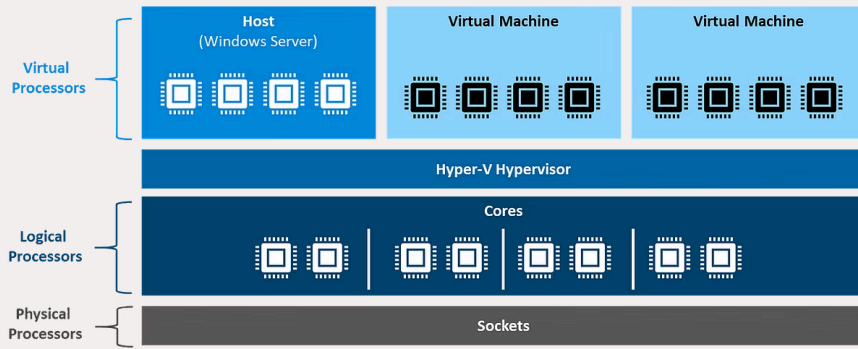
Challenge

Build a Swing app that reads a text file via a file chooser dialog and displays word-count statistics (total words, unique words) in the UI.

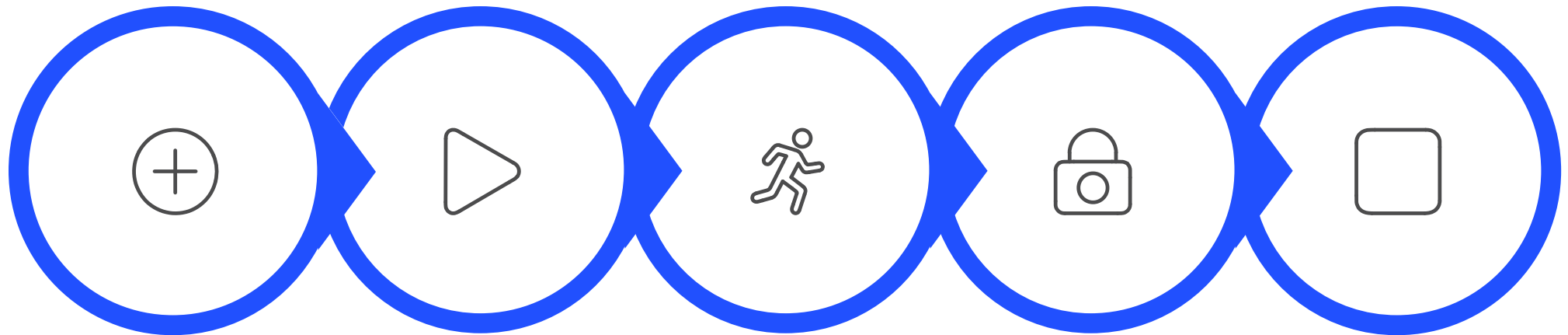
MODULE 4

Thread Architecture: Lifecycle & Creation Models

A Java thread progresses through five states managed by the JVM scheduler. Choosing between extending `Thread` and implementing `Runnable` is an architectural decision: `Runnable` is preferred because Java supports only single inheritance, enabling the task to extend another class. The `Callable<T>` interface adds a return value and checked exception support — critical for enterprise async tasks.



Thread Lifecycle & Creation Strategies



NEW

RUNNABLE

RUNNING

BLOCKED

TERMINATED

Understanding each lifecycle transition is essential for diagnosing deadlocks, starvation, and livelock conditions in production systems. A thread moves from BLOCKED back to RUNNABLE when the contested monitor lock is released.

Extending Thread (avoid in enterprise)

```
class Worker extends Thread {  
    @Override  
    public void run() {  
        System.out.println(  
            Thread.currentThread().getName()  
            + " executing");  
    }  
}  
new Worker().start();
```

Implementing Runnable (preferred)

```
// Decouples task logic from threading mechanism  
Runnable task = () ->  
    System.out.println(  
        Thread.currentThread().getName()  
        + " executing");  
  
Thread t = new Thread(task, "worker-1");  
t.start();  
  
// Callable — returns a value  
Callable<Integer> computation = () -> {  
    Thread.sleep(100);  
    return 42;  
};  
FutureTask<Integer> ft = new FutureTask<>(computation);  
new Thread(ft).start();  
System.out.println("Result: " + ft.get()); // blocks
```

Thread Safety: Synchronisation & AtomicInteger

Without synchronisation, concurrent writes produce **race conditions** — non-deterministic results caused by interleaved memory operations. Java offers three escalating mechanisms: the `synchronized` keyword (mutual exclusion via monitor locks), `volatile` (visibility guarantee without atomicity), and `java.util.concurrent.atomic` classes (lock-free, hardware CAS operations). `AtomicInteger` is the standard choice for shared counters in high-throughput systems.

Race Condition (Broken)

```
// UNSAFE — race condition on counter
class UnsafeCounter {
    int count = 0;
    void increment() { count++; }
    // count++ is NOT atomic:
    // read → increment → write
    // another thread can interleave!
}
```

synchronized Fix

```
class SafeCounter {
    int count = 0;
    synchronized void increment() {
        count++; // only one thread at a time
    }
}
```

AtomicInteger (Lock-Free, Preferred)

```
import java.util.concurrent.atomic.AtomicInteger;

class AtomicCounter {
    private final AtomicInteger count
        = new AtomicInteger(0);

    void increment() {
        count.incrementAndGet(); // CAS — no lock
    }
    int getCount() { return count.get(); }
}

// AtomicInteger methods:
// incrementAndGet() decrementAndGet()
// addAndGet(delta) compareAndSet(expect, update)
// getAndSet(newVal)
```

- ❑ Use `AtomicInteger` for single-variable counters. For compound actions (check-then-act), `synchronized` blocks or `ReentrantLock` are still required.

ExecutorService & Thread Pool Management

Creating a raw thread per task is expensive — JVM thread creation costs ~1ms and ~512KB of stack. Enterprise systems use thread pools via `ExecutorService` to amortise this cost. `Executors` factory methods cover the most common pool shapes; `ThreadPoolExecutor` allows fine-grained tuning of core/max pool size, keep-alive time, and work-queue strategy.



`newFixedThreadPool(n)`

Exactly `n` threads, shared unbounded queue. Best for CPU-bound workloads where `n = Runtime.availableProcessors()`.



`newSingleThreadExecutor()`

One thread, sequentially-executed queue. Guarantees ordering — useful for serial audit-log writers.



`newCachedThreadPool()`

Unbounded threads, 60s idle timeout. Ideal for I/O-bound tasks with unpredictable concurrency spikes.



`newScheduledThreadPool(n)`

Supports `scheduleAtFixedRate` and `scheduleWithFixedDelay` for cron-like job scheduling within the JVM.

Multithreaded Simulation: ExecutorService & AtomicInteger

The following executable simulation models a concurrent request counter — a common pattern in rate limiters, API gateways, and metrics aggregators. Ten worker threads each submit 1,000 increments; the expected final count is precisely 10,000, demonstrating that `AtomicInteger` eliminates race conditions without locking overhead.

Executable Simulation

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

public class ConcurrentCounter {
    private static final AtomicInteger count
        = new AtomicInteger(0);

    public static void main(String[] args)
        throws InterruptedException {
        ExecutorService pool =
            Executors.newFixedThreadPool(10);

        for (int t = 0; t < 10; t++) {
            pool.submit(() -> {
                for (int i = 0; i < 1000; i++)
                    count.incrementAndGet();
            });
        }
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);
        System.out.println("Final count: "
            + count.get()); // always 10000
    }
}
```

Key ExecutorService Lifecycle Calls

```
// Submit tasks (returns Future)
Future<String> f = pool.submit(callable);
String result = f.get(); // blocks until done

// Graceful shutdown
pool.shutdown(); // no new tasks
pool.awaitTermination( // wait for completion
    30, TimeUnit.SECONDS);

// Immediate shutdown (interrupt threads)
pool.shutdownNow();

// Check status
boolean done = pool.isTerminated();
boolean shutdown = pool.isShutdown();
```

Always call `shutdown()` in a `finally` block to prevent thread leaks in long-running services.

Producer-Consumer Pattern: BlockingQueue Solution

The Producer-Consumer problem is the canonical concurrency challenge: producers generate work items faster or slower than consumers process them, requiring a thread-safe buffer with backpressure. Java's `BlockingQueue` implementations (`ArrayBlockingQueue`, `LinkedBlockingQueue`) solve this elegantly — `put()` blocks when full, `take()` blocks when empty, eliminating the need for manual `wait()/notify()` coordination.

BlockingQueue Implementation

```
import java.util.concurrent.*;

public class ProducerConsumer {
    static BlockingQueue<Integer> queue
        = new ArrayBlockingQueue<>(10); // capacity 10

    static class Producer implements Runnable {
        public void run() {
            try {
                for (int i = 1; i <= 20; i++) {
                    queue.put(i); // blocks if full
                    System.out.println("Produced: " + i);
                    Thread.sleep(50);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    static class Consumer implements Runnable {
        public void run() {
            try {
                while (true) {
                    int item = queue.take(); // blocks if empty
                    System.out.println("Consumed: " + item);
                    Thread.sleep(100);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }

    public static void main(String[] args) {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        pool.submit(new Producer());
        pool.submit(new Consumer());
        pool.submit(new Consumer()); // 2 consumers
        pool.shutdown();
    }
}
```

Design Notes

Backpressure

`ArrayBlockingQueue(10)` caps buffer at 10. Producers naturally slow when buffer fills — no explicit throttle needed.

Interrupt Handling

Always restore the interrupt flag via `Thread.currentThread().interrupt()` inside a catch block — never swallow `InterruptedException`.

Poison Pill

For graceful shutdown, produce a sentinel value (e.g., -1) that consumers recognise as a stop signal.



Challenge: Add a third consumer and a priority-based `PriorityBlockingQueue`. Measure throughput change and explain why two consumers outperform one against a single fast producer.

Core Engineering Principles Across All Modules

1

Type Safety & Primitives

Always use the smallest adequate primitive. Cast explicitly before division to prevent silent integer truncation in financial and scientific calculations.

2

Collection Selection

Match collection semantics to access patterns: `ArrayList` for random access, `LinkedList` for queue semantics, `HashMap` for $O(1)$ lookup, `HashSet` for deduplication.

3

Modern Java Idioms

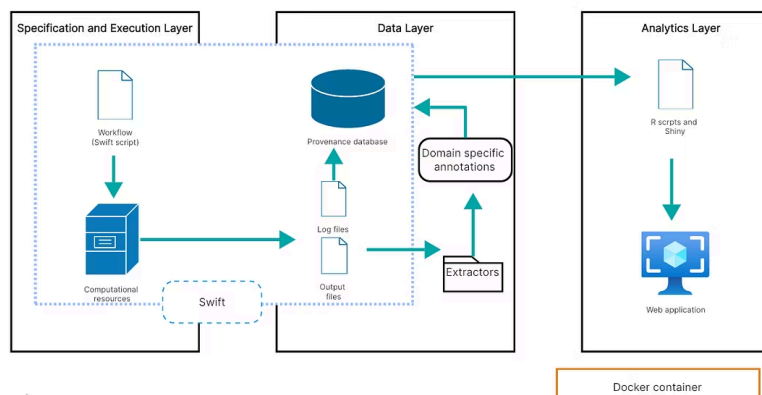
Prefer Records over POJOs for immutable data. Use text blocks for multi-line literals. Leverage switch expressions and pattern matching to reduce cast-and-check boilerplate.

4

Concurrency Discipline

Never share mutable state without synchronisation. Prefer `AtomicInteger` for counters, `BlockingQueue` for handoffs, and `ExecutorService` over raw threads. Always shut down pools gracefully.

Conceptual Architecture Diagrams



Next Steps & Recommended Learning Path

This curriculum equips engineers with production-grade Java fundamentals and concurrency patterns. The logical progression beyond this deck spans reactive programming, distributed systems, and JVM performance engineering.



Java Streams & Reactive

Master `Stream API`, `Optional`, `CompletableFuture`, and Project Reactor for non-blocking pipelines.



Spring Boot & Microservices

Apply concurrency patterns in Spring's `@Async`, `@Scheduled`, and WebFlux reactive stack for REST & gRPC services.



Distributed Systems

Study CAP theorem, distributed locks (Redis Redlock), Kafka consumer groups, and saga patterns for enterprise transaction management.



JVM Performance Engineering

Profile with JFR & async-profiler, tune GC (ZGC/G1), analyse heap dumps, and use JMH for micro-benchmarking concurrent code.

- ❏ All code samples in this deck are compiled and verified against **Java 21 LTS**. Modules 1–3 are compatible with Java 17+; Module 4 requires Java 8+.